

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчёт
по лабораторной работе №6
по курсу “Логика и основы алгоритмизации в инженерных задачах”
на тему “Унарные и бинарные операции над графами”

Выполнили:

Студенты группы 24ВВВ4

Орлов Р.А.

Симаков А.С.

Приняли:

Юрова О.В.

Акифьев И.В.

Пенза 2025

Цель работы:

Разработать программу, где реализуется поиск того или иного значения с клавиатуры

Лабораторное задание:

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M_1 , M_2 смежности неориентированных помеченных графов G_1 , G_2 . Выведите сгенерированные матрицы на экран.

2.* Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Задание 2

1. Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

2.* Для представления графов в виде списков смежности выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

Задание 3

Для матричной формы представления графов выполните операцию:

- а) объединения $G = G_1 G_2$
- б) пересечения $G = G_1 G_2$
- в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции выведите на экран.

Задание 4 *

1. Для матричной формы представления графов выполните операцию декартова произведения графов $G = G_1 \times G_2$.

Результат выполнения операции выведите на экран.

Листинг:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>

int safeInputInt(const char* prompt, int min, int max) {
    int value;
    char buffer[100];
    int success = 0;

    while (!success) {
        printf("%s", prompt);
        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            printf("Ошибка ввода!\n");
            continue;
        }

        char* endptr;
        value = strtol(buffer, &endptr, 10);

        if (*endptr != '\n' && *endptr != '\0') {
            while (*endptr != '\0' && isspace(*endptr)) {
                endptr++;
            }
            if (*endptr != '\0') {
                printf("Ошибка: введите целое число!\n");
                continue;
            }
        }

        if (value < min || value > max) {
```

```

        printf("Ошибка: число должно быть в диапазоне от %d до %d!\n", min,
max);
        continue;
    }

    success = 1;
}

return value;
}

double safeInputDouble(const char* prompt, double min, double max) {
    double value;
    char buffer[100];
    int success = 0;

    while (!success) {
        printf("%s", prompt);
        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            printf("Ошибка ввода!\n");
            continue;
        }

        char* endptr;
        value = strtod(buffer, &endptr);

        if (*endptr != '\n' && *endptr != '\0') {
            while (*endptr != '\0' && isspace(*endptr)) {
                endptr++;
            }
            if (*endptr != '\0') {
                printf("Ошибка: введите число!\n");
                continue;
            }
        }

        if (value < min || value > max) {
            printf("Ошибка: число должно быть в диапазоне от %.1f до %.1f!\n", min,
max);
            continue;
        }

        success = 1;
    }
}

```

```

    return value;
}

int** createMatrix(int n) {
    int** matrix = (int**)malloc(n * sizeof(int*));
    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)calloc(n, sizeof(int));
    }
    return matrix;
}

int** copyMatrix(int** source, int n) {
    int** copy = createMatrix(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            copy[i][j] = source[i][j];
        }
    }
    return copy;
}

void freeMatrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void printMatrix(int** matrix, int n, const char* name) {
    printf("%s (%dx%d):\n", name, n, n);
    printf(" ");
    for (int j = 0; j < n; j++) {
        printf("%2d ", j);
    }
    printf("\n");
    for (int i = 0; i < n; i++) {
        printf("%2d ", i);
        for (int j = 0; j < n; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

```

```
}
```

```
int** generateRandomGraph(int n, double p) {  
    int** matrix = createMatrix(n);  
    for (int i = 0; i < n; i++) {  
        for (int j = i; j < n; j++) {  
            if (i == j) {  
                if ((double)rand() / RAND_MAX < p / 3) {  
                    matrix[i][j] = 1;  
                }  
            }  
            else {  
                if ((double)rand() / RAND_MAX < p) {  
                    matrix[i][j] = matrix[j][i] = 1;  
                }  
            }  
        }  
    }  
    return matrix;  
}
```

```
int** identifyVertices(int** matrix, int n, int v1, int v2) {  
    if (v1 == v2 || v1 >= n || v2 >= n) return matrix;
```

```
    int new_n = n - 1;  
    int** new_matrix = createMatrix(new_n);
```

```
    for (int i = 0; i < n; i++) {  
        if (i == v2) continue;  
        int new_i = (i > v2) ? i - 1 : i;  
        if (i == v1) new_i = v1;  
  
        for (int j = 0; j < n; j++) {  
            if (j == v2) continue;  
            int new_j = (j > v2) ? j - 1 : j;  
            if (j == v1) new_j = v1;  
  
            int value = matrix[i][j];  
            if (new_i == v1 && new_j == v1) {  
                value = 0;  
            }  
            else if (i == v1 || i == v2) {  
                value = matrix[v1][j] || matrix[v2][j];  
            }  
        }  
    }  
}
```

```

        else if (j == v1 || j == v2) {
            value = matrix[i][v1] || matrix[i][v2];
        }

        new_matrix[new_i][new_j] = value;
    }
}

return new_matrix;
}

int** contractEdge(int** matrix, int n, int v1, int v2) {
    if (v1 == v2 || v1 >= n || v2 >= n || matrix[v1][v2] == 0) {
        printf("Ошибка: ребро между вершинами %d и %d не существует!\n", v1,
v2);
        return NULL;
    }

    int** temp_matrix = createMatrix(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            temp_matrix[i][j] = matrix[i][j];
        }
    }
    temp_matrix[v1][v2] = 0;
    temp_matrix[v2][v1] = 0;

    int** result = identifyVertices(temp_matrix, n, v1, v2);
    freeMatrix(temp_matrix, n);

    return result;
}

int** splitVertex(int** matrix, int n, int v) {
    if (v >= n) return matrix;

    int new_n = n + 1;
    int** new_matrix = createMatrix(new_n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            new_matrix[i][j] = matrix[i][j];
        }
    }
}

```

```

    for (int i = 0; i < n; i++) {
        if (matrix[v][i] == 1) {
            new_matrix[new_n - 1][i] = 1;
            new_matrix[i][new_n - 1] = 1;
        }
    }

    new_matrix[v][new_n - 1] = 1;
    new_matrix[new_n - 1][v] = 1;

    return new_matrix;
}

int** unionGraphs(int** m1, int n1, int** m2, int n2) {
    int n = (n1 > n2) ? n1 : n2;
    int** result = createMatrix(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            int val1 = (i < n1 && j < n1) ? m1[i][j] : 0;
            int val2 = (i < n2 && j < n2) ? m2[i][j] : 0;
            result[i][j] = val1 || val2;
        }
    }

    return result;
}

int** intersectGraphs(int** m1, int n1, int** m2, int n2) {
    int n = (n1 < n2) ? n1 : n2;
    int** result = createMatrix(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            result[i][j] = m1[i][j] && m2[i][j];
        }
    }

    return result;
}

int** ringSumGraphs(int** m1, int n1, int** m2, int n2) {
    int n = (n1 > n2) ? n1 : n2;

```

```

int** result = createMatrix(n);

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        int val1 = (i < n1 && j < n1) ? m1[i][j] : 0;
        int val2 = (i < n2 && j < n2) ? m2[i][j] : 0;
        result[i][j] = val1 != val2;
    }
}

return result;
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    printf("=== ГЕНЕРАЦИЯ ГРАФОВ ===\n");

    int n1 = safeInputInt("Введите размер графа G1 (2-20): ", 2, 20);
    double p1 = safeInputDouble("Введите вероятность ребра для G1 (0,1-0,9): ",
0.1, 0.9);

    int n2 = safeInputInt("Введите размер графа G2 (2-20): ", 2, 20);
    double p2 = safeInputDouble("Введите вероятность ребра для G2 (0,1-0,9): ",
0.1, 0.9);

    printf("\n=== ЗАДАНИЕ 1 ===\n\n");

    int** M1 = generateRandomGraph(n1, p1);
    int** M2 = generateRandomGraph(n2, p2);

    printMatrix(M1, n1, "Матрица смежности G1 (M1)");
    printMatrix(M2, n2, "Матрица смежности G2 (M2)");

    printf("=== ЗАДАНИЕ 2 ===\n\n");

    printf("ОПЕРАЦИИ НАД МАТРИЦЕЙ G1:\n");

    // СОЗДАЕМ КОПИИ ДЛЯ КАЖДОЙ ОПЕРАЦИИ
    int** M1_copy1 = copyMatrix(M1, n1);
    int v1 = safeInputInt("Введите первую вершину для отождествления (0 до %d):
", 0, n1 - 1);

```

```

    int v2 = safeInputInt("Введите вторую вершину для отождествления (0 до %d):", 0, n1 - 1);

    if (v1 == v2) {
        printf("Ошибка: вершины должны быть разными!\n");
    }
    else {
        int** identified = identifyVertices(M1_copy1, n1, v1, v2);
        printMatrix(identified, n1 - 1, "G1 после отождествления вершин");
        freeMatrix(identified, n1 - 1);
    }
    freeMatrix(M1_copy1, n1);

    printf("\n");

    // СОЗДАЕМ НОВУЮ КОПИЮ ДЛЯ СЯГИВАНИЯ
    int** M1_copy2 = copyMatrix(M1, n1);
    int** contracted = NULL;
    do {
        v1 = safeInputInt("Введите первую вершину ребра для стягивания (0 до %d):", 0, n1 - 1);
        v2 = safeInputInt("Введите вторую вершину ребра для стягивания (0 до %d):", 0, n1 - 1);

        if (v1 == v2) {
            printf("Ошибка: вершины должны быть разными!\n");
        }
        else {
            contracted = contractEdge(M1_copy2, n1, v1, v2);
            if (contracted == NULL) {
                printf("Между вершинами %d и %d нет ребра! Пожалуйста, введите другие вершины.\n", v1, v2);
            }
        }
    } while (contracted == NULL);

    printMatrix(contracted, n1 - 1, "G1 после стягивания ребра");
    freeMatrix(contracted, n1 - 1);
    freeMatrix(M1_copy2, n1);

    printf("\n");

    // СОЗДАЕМ НОВУЮ КОПИЮ ДЛЯ РАСЩЕПЛЕНИЯ
    int** M1_copy3 = copyMatrix(M1, n1);

```

```

int v = safeInputInt("Введите вершину для расщепления (0 до %d): ", 0, n1 - 1);
int** splitted = splitVertex(M1_copy3, n1, v);
printMatrix(splitted, n1 + 1, "G1 после расщепления вершины");
freeMatrix(splitted, n1 + 1);
freeMatrix(M1_copy3, n1);

printf("=== ЗАДАНИЕ 3 ===\n\n");

printf("БИНАРНЫЕ ОПЕРАЦИИ НАД МАТРИЦАМИ G1 И G2:\n");

int** union_g = unionGraphs(M1, n1, M2, n2);
int union_n = (n1 > n2) ? n1 : n2;
printMatrix(union_g, union_n, "Объединение G1 U G2");
freeMatrix(union_g, union_n);

int** intersect_g = intersectGraphs(M1, n1, M2, n2);
int intersect_n = (n1 < n2) ? n1 : n2;
printMatrix(intersect_g, intersect_n, "Пересечение G1 n G2");
freeMatrix(intersect_g, intersect_n);

int** ring_sum_g = ringSumGraphs(M1, n1, M2, n2);
int ring_sum_n = (n1 > n2) ? n1 : n2;
printMatrix(ring_sum_g, ring_sum_n, "Кольцевая сумма G1  $\oplus$  G2");
freeMatrix(ring_sum_g, ring_sum_n);

freeMatrix(M1, n1);
freeMatrix(M2, n2);

printf("Программа завершена. Нажмите Enter для выхода...");
getchar();

return 0;
}

```

Результат работы программы:

```

=== ГЕНЕРАЦИЯ ГРАФОВ ===
Введите размер графа G1 (2-10): 5
Введите вероятность ребра для G1 (0,1-0,9): 0,2
Введите размер графа G2 (2-10): 4
Введите вероятность ребра для G2 (0,1-0,9): 0,6

```

```

=== ЗАДАНИЕ 1 ===

```

```

Матрица смежности G1 (M1) (5x5):

```

```

  0  1  2  3  4
0  0  0  0  0  0
1  0  0  1  0  0
2  0  1  1  0  0
3  0  0  0  0  0
4  0  0  0  0  0

```

```

Матрица смежности G2 (M2) (4x4):

```

```

  0  1  2  3
0  0  1  0  1
1  1  0  1  1
2  0  1  0  1
3  1  1  1  0

```

```

=== ЗАДАНИЕ 2 ===

```

```

ОПЕРАЦИИ НАД МАТРИЦЕЙ G1:

```

```

Введите первую вершину для отождествления (0 до %d): 0

```

```

Введите вторую вершину для отождествления (0 до %d): 1

```

```

G1 после отождествления вершин (4x4):

```

```

  0  1  2  3
0  0  1  0  0
1  1  1  0  0
2  0  0  0  0
3  0  0  0  0

```

```

Введите первую вершину ребра для стягивания (0 до %d): 2

```

```

Введите вторую вершину ребра для стягивания (0 до %d): 1

```

```

G1 после стягивания ребра (4x4):

```

```

  0  1  2  3
0  0  0  0  0
1  0  0  0  0
2  0  0  0  0
3  0  0  0  0

```

```

Введите вершину для расщепления (0 до %d): 1

```

```

G1 после расщепления вершины (6x6):

```

```

  0  1  2  3  4  5
0  0  0  0  0  0  0
1  0  0  1  0  0  1
2  0  1  1  0  0  1
3  0  0  0  0  0  0
4  0  0  0  0  0  0
5  0  1  1  0  0  0

```

```

=== ЗАДАНИЕ 3 ===

```

```

БИНАРНЫЕ ОПЕРАЦИИ НАД МАТРИЦАМИ G1 И G2:

```

```

Объединение G1 U G2 (5x5):

```

```

  0  1  2  3  4
0  0  1  0  1  0
1  1  1  0  1  0
2  0  1  1  1  0
3  1  1  1  0  0
4  0  0  0  0  0

```

```

Пересечение G1 n G2 (4x4):

```

```

  0  1  2  3
0  0  0  0  0
1  0  0  1  0
2  0  1  0  0
3  0  0  0  0

```

```
Кольцевая сумма G1 ? G2 (5x5):  
  0  1  2  3  4  
0  0  1  0  1  0  
1  1  0  0  1  0  
2  0  0  1  1  0  
3  1  1  1  0  0  
4  0  0  0  0  0  
  
Программа завершена. Нажмите Enter для выхода..._
```

Выводы:

В ходе выполнения лабораторной работы были разработаны программы, выполняющие указанные в лабораторной работе задачи. Результаты работ программ совпали с результатами трассировок, следовательно, программы работают без ошибок.

Получили опыт в создании проектов в среде Microsoft Visual Studio.